

Moneyora

Personal Finance & AI Budget Planner

Plan. Track. Thrive.

SYSTEM DESIGN DOCUMENT (SDD)

Document Version:	1.0 — Final
Date:	21 June 2026
Based On:	SRS v1.0 (Approved)
Architecture Style:	Clean Architecture — Offline-First Hybrid
Platform:	Flutter — iOS 14.0+ & Android 8.0+ (API 26+)
State Management:	Riverpod 2.x
Local Database:	SQLite + SQLCipher (AES-256)
AI Strategy:	On-Device (ML Kit) + Optional Cloud API
Status:	READY FOR IMPLEMENTATION

Developer: Sanduni • Individual Project

System Design Document • Phase 2 of Development Lifecycle

TABLE OF CONTENTS

1. INTRODUCTION & DOCUMENT SCOPE

- 1.1 Purpose of this Document
- 1.2 Relationship to SRS
- 1.3 Design Goals & Principles

2. SYSTEM ARCHITECTURE

- 2.1 Architecture Style — Clean Architecture
- 2.2 Offline-First Hybrid Model
- 2.3 Architecture Layers Explained
- 2.4 Key Design Decisions

3. PROJECT STRUCTURE

- 3.1 Flutter Project Folder Structure
- 3.2 Feature Module Structure
- 3.3 Dependency Injection Strategy

4. SCREEN & NAVIGATION DESIGN

- 4.1 Screen Inventory
- 4.2 Navigation Flow Diagram
- 4.3 Screen Descriptions

5. DATA DESIGN

- 5.1 Local Database Design
- 5.2 Data Access Patterns
- 5.3 Database Migration Strategy
- 5.4 Encryption Implementation

6. STATE MANAGEMENT DESIGN

- 6.1 Riverpod Architecture
- 6.2 Provider Hierarchy
- 6.3 State Flow Pattern

7. AI FEATURE DESIGN

- 7.1 Money Plan Generator — Detailed Design
- 7.2 Receipt Scanner — Detailed Design
- 7.3 AI Integration & Fallback Strategy

8. SECURITY DESIGN

- 8.1 Data Encryption
- 8.2 Authentication Flow
- 8.3 Secure API Communication

9. COMPONENT INTERFACES

- 9.1 Repository Interface Contracts
- 9.2 Use Case Interface Contracts
- 9.3 External API Interfaces

10. TECHNOLOGY STACK DETAILS

- 10.1 Core Dependencies
- 10.2 Package Versions & Configuration
- 10.3 Build & Environment Setup

1. INTRODUCTION & DOCUMENT SCOPE

1.1 Purpose of this Document

This System Design Document (SDD) defines the complete technical architecture, component design, data flows, and implementation blueprint for **Moneyora** — a personal finance and AI-powered budget planning mobile application. It translates the approved SRS v1.0 requirements into actionable technical specifications that guide all development activities.

1.2 Relationship to SRS

Document	Purpose
SRS v1.0	Defines WHAT the system must do — functional and non-functional requirements
SDD v1.0 (this doc)	Defines HOW the system will be built — architecture, components, data flows
Sprint Implementation	Defines WHEN each component is built — sprint-by-sprint delivery

1.3 Design Goals & Principles

- **Offline-First:** All core features work without internet. Online features degrade gracefully.
- **Clean Architecture:** Strict layer separation ensures testability and maintainability.
- **Security by Design:** AES-256 encryption from day one — not added later.
- **Feature Modularity:** Each feature is self-contained — can be developed and tested independently.
- **AI as Enhancement:** AI features improve the experience but are never required for basic use.
- **Performance First:** Sub-2s cold start, sub-100ms DB queries — built into design, not optimized after.
- **Single Codebase:** Flutter ensures identical behavior across iOS and Android from one codebase.

2. SYSTEM ARCHITECTURE

2.1 Architecture Style — Clean Architecture

Moneyora adopts **Clean Architecture** (Robert C. Martin) with a feature-first folder organization. The core principle is that **dependencies only point inward** — the Domain layer has zero dependencies on Flutter, databases, or external packages. This makes business logic fully testable in pure Dart.

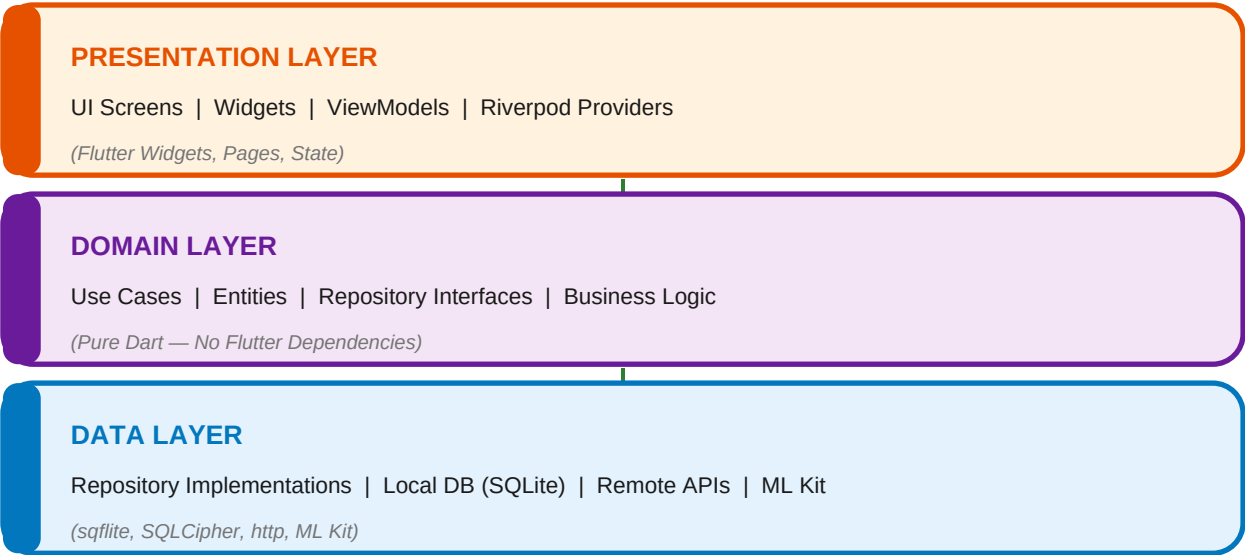
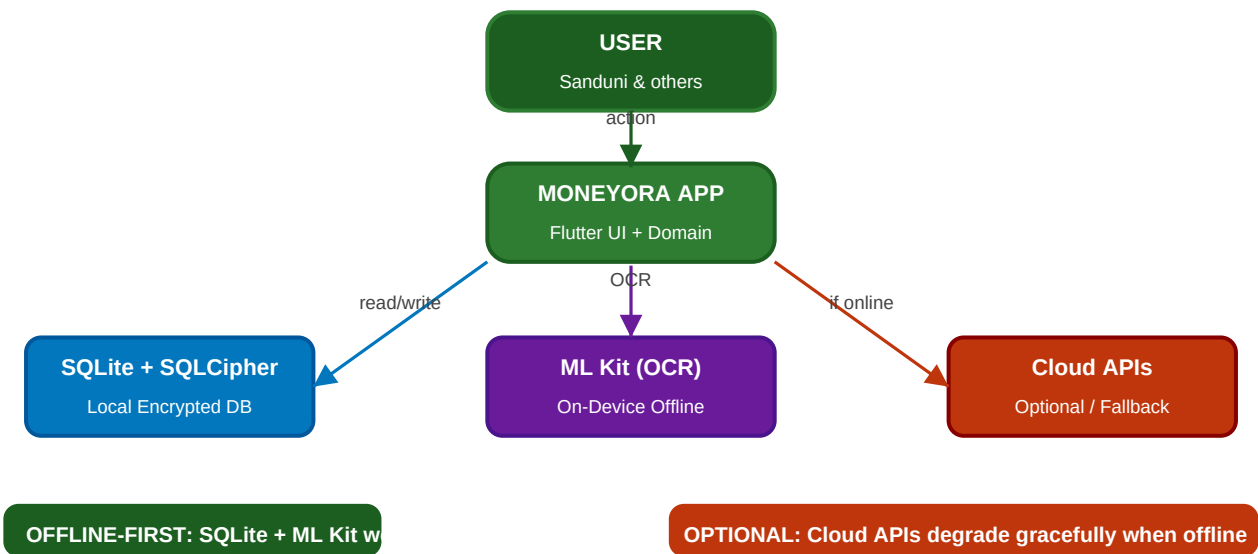


Figure 1: Clean Architecture — Dependency flows inward (Presentation depends on Domain; Data depends on Domain)

2.2 Offline-First Hybrid Model

The system operates in a hybrid connectivity model. The primary operating mode is fully offline — all transactions, analytics, and budget planning work without any internet connection. Optional online capabilities (cloud AI enhancement, exchange rates, cloud backup) activate automatically when connectivity is available and degrade gracefully when not.



2.3 Architecture Layers Explained

Layer	Contains	Technology	Responsibility
Presentation Layer	Flutter Widgets, Pages, ViewModels	Riverpod Providers + StateNotifier	Displays data, captures user input, delegates all logic to domain layer via use cases
Domain Layer	Entities, Use Cases, Repository Interfaces	Pure Dart — zero external deps	Contains all business rules. Defines what the app CAN do. Completely independent of Flutter or any framework
Data Layer	Repository Implementations, Data Sources, Models	sqlite, SQLCipher, http, ML Kit	Implements the repository interfaces defined in Domain. Handles DB, network, and device hardware access

2.4 Key Design Decisions

Why Flutter?

Single codebase for iOS + Android. Strong community, excellent package ecosystem, and native performance via Dart compilation. Reduces development time by ~40% vs separate native apps.

Why Riverpod for State Management?

Riverpod 2.x is compile-safe (no runtime errors from missing providers), fully testable, and supports async state with AsyncValue out of the box — ideal for database and AI operations.

Why SQLite + SQLCipher?

SQLite is the standard for offline mobile databases. SQLCipher adds transparent AES-256 encryption with zero performance overhead, meeting NFR-SEC-001 without code changes.

Why Google ML Kit for OCR?

ML Kit Text Recognition v2 runs entirely on-device, requires no API key, and supports both iOS and Android. It handles the offline OCR requirement (FR-RCP-004) at zero cost.

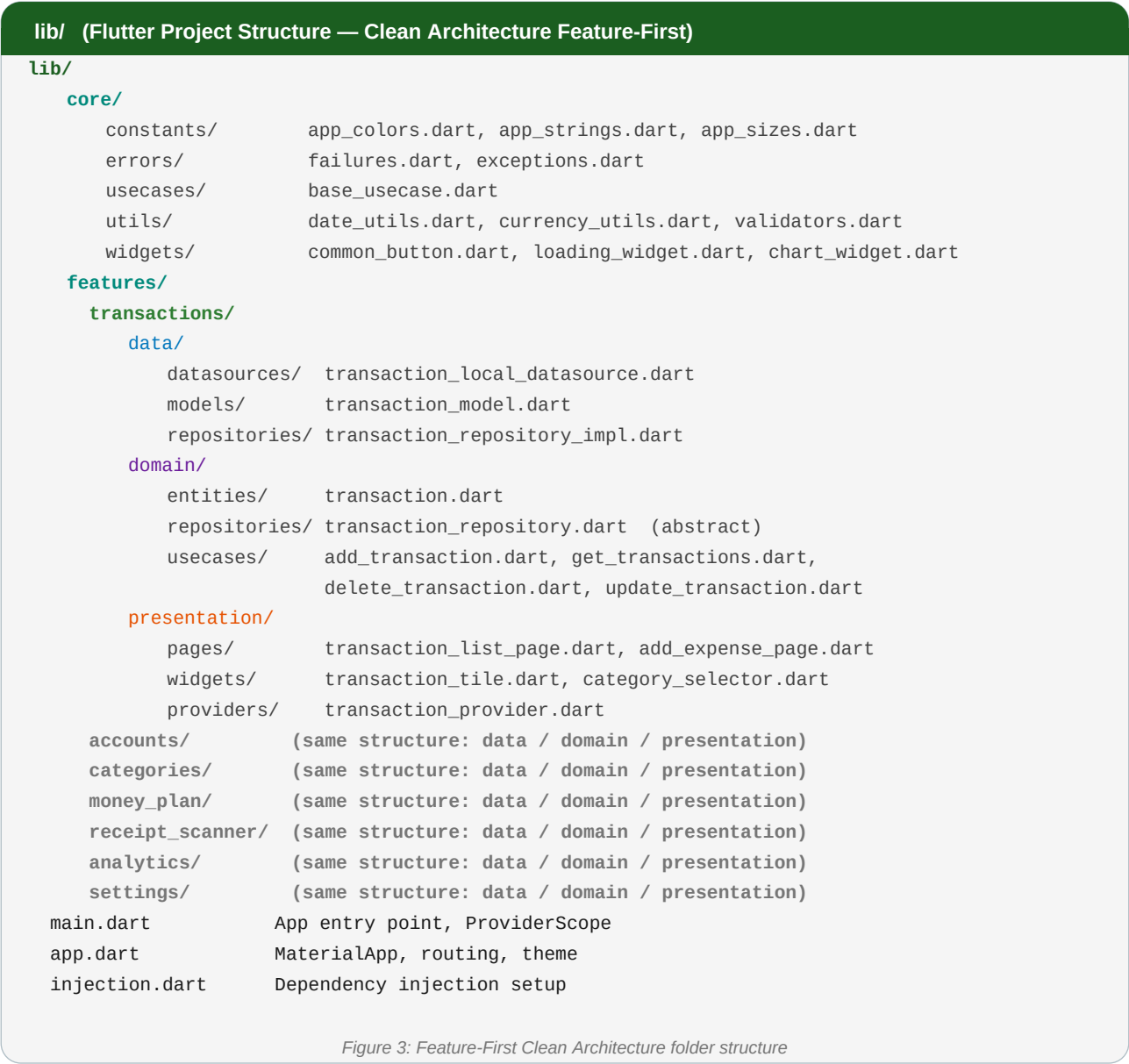
Why Optional Cloud AI?

Using optional AI APIs (Gemini/OpenAI) rather than required ones keeps the app fully functional offline while allowing enhanced suggestions when internet is available.

3. PROJECT STRUCTURE

3.1 Flutter Project Folder Structure

The project follows a **Feature-First Clean Architecture** structure. Each feature (transactions, accounts, money_plan, etc.) is a self-contained module with its own data, domain, and presentation sub-folders.



data/models/	Data Transfer Objects (DTOs) — extend Entities, add fromJson/toMap/fromMap methods
data/repositories/	Concrete implementation of the domain repository interface
domain/entities/	Pure Dart business objects — no Flutter, no JSON, no DB dependencies
domain/repositories/	Abstract interface defining what data operations are available
domain/usecases/	One class per business operation — each use case calls one repository method
presentation/pages/	Full screen widgets — composed of smaller widgets
presentation/widgets/	Reusable UI components specific to this feature
presentation/providers/	Riverpod providers and StateNotifiers for this feature's state

3.3 Dependency Injection Strategy

Moneyora uses **Riverpod Providers** as the dependency injection mechanism — no separate DI framework needed. Dependencies are wired at the provider level:

```
// injection.dart – Provider wiring example

// Data Layer
final databaseProvider = Provider((ref) => DatabaseHelper.instance.db);
final transactionDatasourceProvider = Provider(
  (ref) => TransactionLocalDatasourceImpl(ref.read(databaseProvider)),
);

// Domain Layer
final transactionRepositoryProvider = Provider(
  (ref) => TransactionRepositoryImpl(ref.read(transactionDatasourceProvider)),
);

// Use Cases
final addTransactionProvider = Provider(
  (ref) => AddTransaction(ref.read(transactionRepositoryProvider)),
);

// Presentation
final transactionNotifierProvider =
  StateNotifierProvider>>(
    (ref) => TransactionNotifier(ref.read(addTransactionProvider)),
  );
```


4. SCREEN & NAVIGATION DESIGN

4.1 Screen Inventory

Screen ID	Screen Name	Key Elements
SCR-001	Home Screen	Main donut chart, balance bar, category icons, FABs
SCR-002	Add Expense Screen	Custom keypad, category row, account selector, note field
SCR-003	Add Income Screen	Same as add expense with income categories
SCR-004	New Transfer Screen	From/To account selectors, amount, date, note
SCR-005	Transaction List Screen	Date-grouped list with filters, search
SCR-006	Transaction Detail Screen	Full detail view + edit + delete
SCR-007	Categories Manager	List of expense/income categories, add/edit/delete
SCR-008	Accounts Manager	Account list with balances, add/edit/archive
SCR-009	Analytics Screen	Income vs expense charts, trend lines, calendar heatmap
SCR-010	Money Plan Home	List of saved plans, create new plan button
SCR-011	Create Plan Wizard	Step 1: period select → Step 2: review allocations → Step 3: confirm
SCR-012	Active Plan Tracker	Real-time actual vs planned per category with progress bars
SCR-013	Scan Receipt Screen	Camera viewfinder, gallery button, scan trigger
SCR-014	Receipt Review Screen	Parsed items list, category selectors, confirm/discard
SCR-015	Receipt History Screen	Searchable list of past scans with thumbnails
SCR-016	Settings Screen	All settings grouped: General, Security, Notifications, Backup
SCR-017	Backup & Restore Screen	Create/restore/schedule backup, cloud sync toggle
SCR-018	Currencies Screen	Multi-currency config and exchange rates
SCR-019	Search Screen	Global transaction search with filters
SCR-020	Passcode Screen	PIN entry/setup and biometric auth prompt

4.2 Navigation Flow Diagram

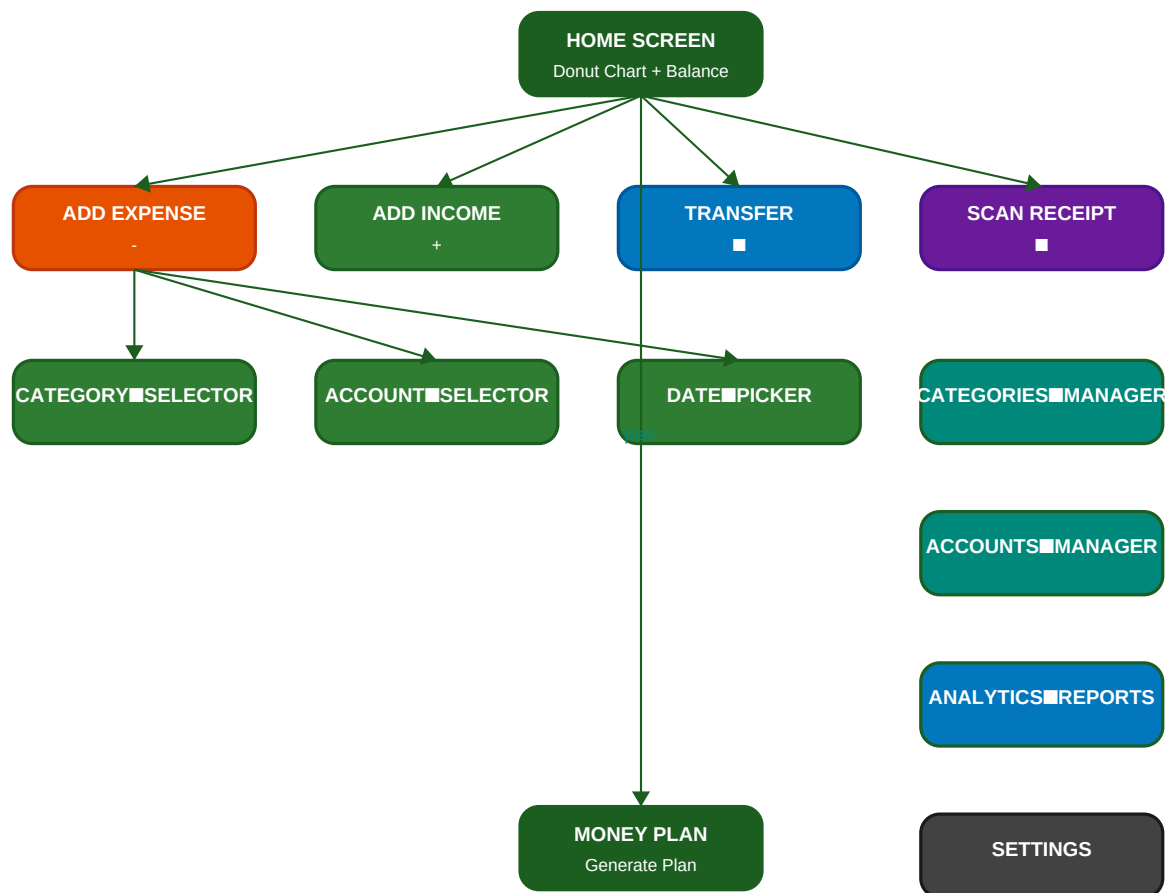


Figure 4: Screen Navigation Flow — arrows indicate navigation direction

4.3 Navigation Implementation

- **Router:** go_router package — declarative routing with deep link support
- **Side Drawers:** Custom slide-in panels for Categories, Accounts, Currencies, Settings
- **Modals:** Bottom sheets for quick actions (date picker, category picker, account picker)
- **FABs:** Red minus FAB (expense) and Green plus FAB (income) always visible on home
- **Back Navigation:** Standard iOS/Android back gesture support throughout
- **Auth Gate:** Passcode/biometric screen shown on app resume if security enabled

5. DATA DESIGN

5.1 Local Database Design

All data is stored in a single **SQLCipher-encrypted SQLite** database file at: `app_documents/moneyora.db` (iOS) / `app_data/moneyora.db` (Android). The database is opened with AES-256 encryption on every app launch.

Database Helper Pattern

```
class DatabaseHelper {
    static final DatabaseHelper instance = DatabaseHelper._init();
    static Database? _database;

    Future get db async {
        _database ??= await _initDB('moneyora.db');
        return _database!;
    }

    Future _initDB(String fileName) async {
        final dbPath = await getDatabasesPath();
        final path = join(dbPath, fileName);
        return await openDatabase(
            path,
            version: 1,
            password: await _getEncryptionKey(), // AES-256 via SQLCipher
            onCreate: _createTables,
            onUpgrade: _migrateTables,
        );
    }
}
```

Index Strategy

Index Name	Columns	Purpose
idx_transactions_date	transactions(date)	Date range queries for charts and filters
idx_transactions_account	transactions(account_id)	Account-specific transaction lists
idx_transactions_category	transactions(category_id)	Category analytics and plan tracking
idx_transactions_type_date	transactions(type, date)	Expense/income separation with date filter
idx_plan_allocations_plan	plan_allocations(plan_id)	Plan detail lookups
idx_receipt_items_scan	receipt_items(receipt_scan_id)	Receipt batch queries
idx_keyword_dict_keyword	keyword_dictionary(keyword)	OCR categorization speed

5.2 Data Access Patterns

Query Type	SQL Pattern
Home Screen Load	SELECT t.*, c.name, c.icon, c.color FROM transactions t JOIN categories c ON t.category_id = c.id WHERE t.account_id = ? AND t.date BETWEEN ? AND ? ORDER BY t.date DESC
Money Plan Analysis	SELECT category_id, AVG(amount) as avg, STDDEV(amount) as std, COUNT(*) as cnt FROM transactions WHERE type='expense' AND date >= date('now','-6 months') GROUP BY category_id
Category Totals for Chart	SELECT c.name, c.color, c.icon, SUM(t.amount) as total FROM transactions t JOIN categories c ON t.category_id = c.id WHERE t.type='expense' AND t.date BETWEEN ? AND ? GROUP BY t.category_id ORDER BY total DESC
Plan vs Actual Tracking	SELECT pa.category_id, pa.allocated_amount, SUM(t.amount) as spent FROM plan_allocations pa LEFT JOIN transactions t ON pa.category_id = t.category_id AND t.date BETWEEN ? AND ? WHERE pa.plan_id = ? GROUP BY pa.category_id

5.3 Database Migration Strategy

- Each schema change increments the database **version number** in DatabaseHelper
- The **onUpgrade** callback handles all migrations with ALTER TABLE / CREATE TABLE statements
- Migration scripts are stored in lib/core/database/migrations/ — one file per version
- Migrations are **additive only** (never drop columns) to protect existing user data
- A **schema_version** table tracks applied migrations for complex multi-step upgrades

5.4 Encryption Implementation

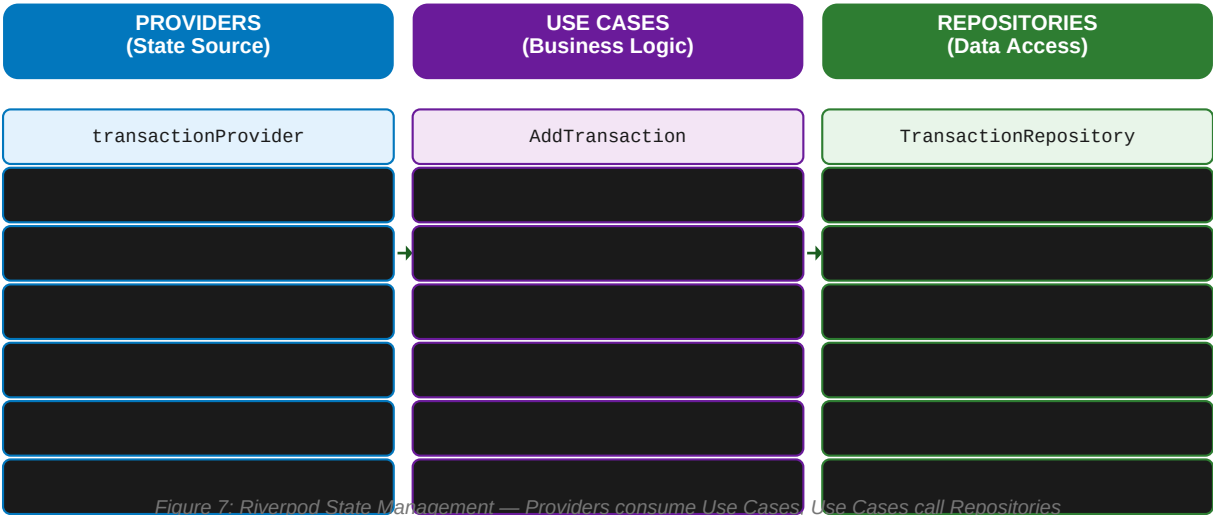
```
// SQLCipher key derivation – stored in Flutter Secure Storage
final secureStorage = FlutterSecureStorage();

Future _getEncryptionKey() async {
  String? key = await secureStorage.read(key: 'db_encryption_key');
  if (key == null) {
    // Generate new 256-bit key on first launch
    key = base64.encode(List.generate(32, (i) => Random.secure().nextInt(256)));
    await secureStorage.write(key: 'db_encryption_key', value: key);
  }
  return key;
}
// Key stored in iOS Keychain / Android Keystore – never in plain text
```

6. STATE MANAGEMENT DESIGN

6.1 Riverpod Architecture

Moneyora uses **Riverpod 2.x** for all state management. Riverpod provides compile-safe dependency injection, testable providers, and reactive state with **AsyncValue** for seamless loading/error/data states throughout the app.



6.2 Provider Hierarchy

Provider Type	Used For	When to Use
Provider	DatabaseHelper, Datasources, Repositories	Singleton dependencies — created once
StateNotifierProvider	TransactionNotifier, PlanNotifier, ScanNotifier	Mutable state with methods
FutureProvider	loadAnalytics, loadTransactions	One-time async data fetching
StreamProvider	watchTransactions, watchPlanProgress	Reactive real-time data streams
NotifierProvider	SettingsNotifier, ThemeNotifier	Simple mutable settings state

6.3 State Flow Pattern

```
// Complete state flow example – Add Expense

// 1. User taps Confirm on Add Expense screen
// 2. Widget calls provider method:
ref.read(transactionNotifierProvider.notifier).addExpense(transaction);

// 3. StateNotifier executes use case:
class TransactionNotifier extends StateNotifier>> {
  final AddTransaction _addTransaction;
  Future addExpense(Transaction t) async {
```

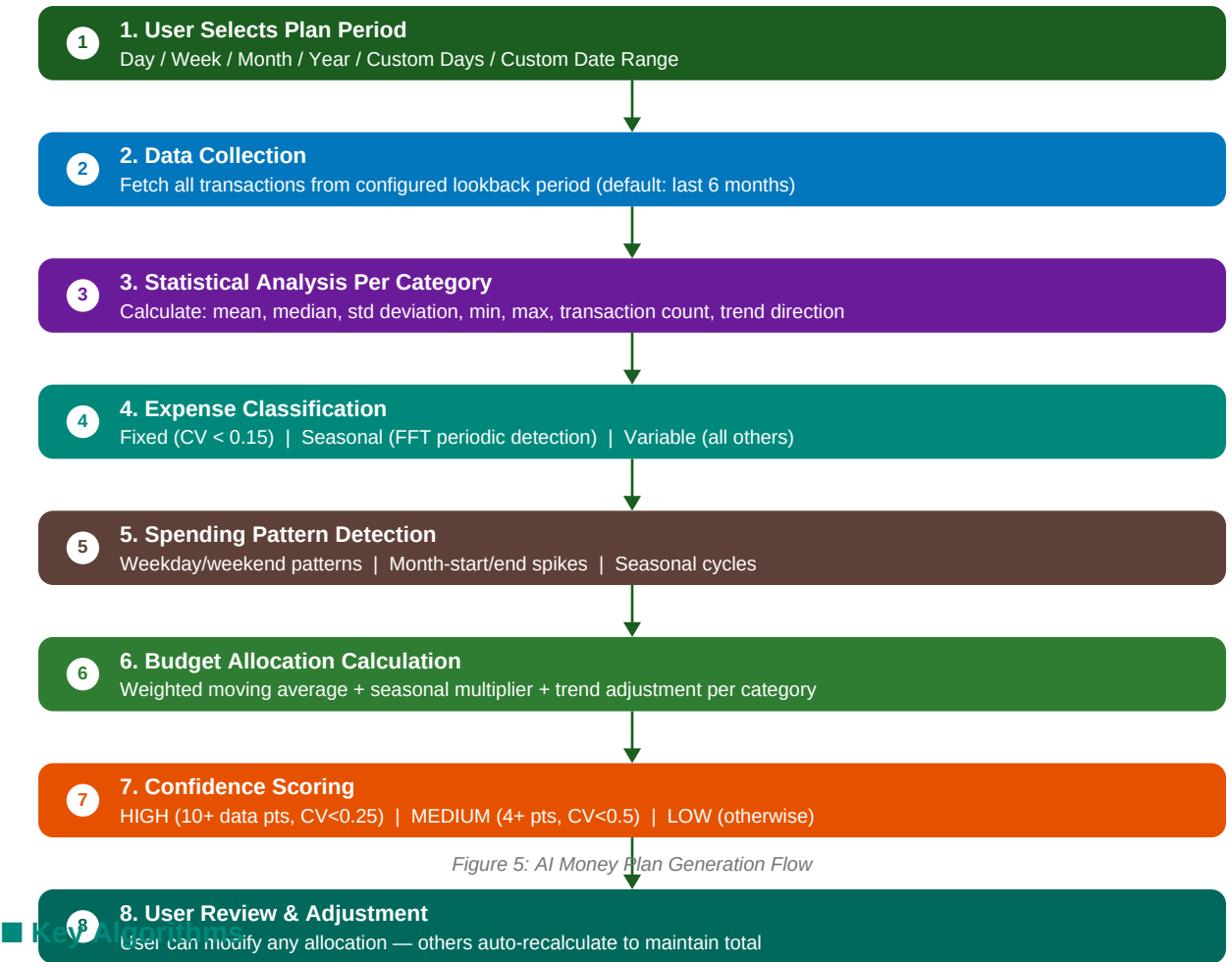
```
state = const AsyncValue.loading();
final result = await _addTransaction(t); // Domain Use Case
result.fold(
  (failure) => state = AsyncValue.error(failure, StackTrace.current),
  (success) => _refreshTransactions(), // Reload from DB
);
}
}

// 4. UI reacts automatically via Consumer widget – no manual refresh needed
```

7. AI FEATURE DESIGN

7.1 Money Plan Generator — Detailed Design

The Money Plan Generator is a pure Dart statistical engine in the Domain layer. It requires no internet connection and produces budget plans from local transaction history. An optional cloud AI call can enhance the plan with natural language suggestions.



Weighted Moving Average

```
recent_avg = mean(last_3_months_amounts)
older_avg = mean(months_4_to_N_amounts)
allocation = (recent_avg * 0.60) + (older_avg * 0.40)
```

Expense Classification

```
CV = std_deviation / mean
Fixed: CV < 0.15 (very consistent)
Seasonal: FFT detects periodic spike
Variable: all others
```

Trend Detection

```
slope = linear_regression(monthly_totals_over_time)
if slope > threshold: add 8% buffer
if slope < -threshold: reduce 5%
```

Confidence Scoring

```
HIGH: data_points >= 10 AND CV < 0.25
MEDIUM: data_points >= 4 AND CV < 0.50
LOW: insufficient data or high variance
```

Daily Allowance

```
daily = category_allocation / plan_days
Displayed as: 'You can spend Rs.220/day on Food'
```

7.2 Receipt Scanner — Detailed Design

The Receipt Scanner pipeline combines on-device OCR (Google ML Kit) with a local keyword-based categorization engine. All processing is offline-capable.

Steps 1-5: 100% OFFLINE (ML Kit on-device)

Steps 6-8: User interaction + local save

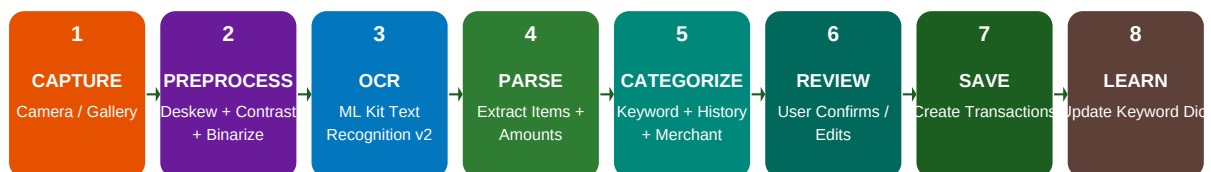


Figure 6: Receipt Scanner Pipeline — all OCR processing happens on-device

■ Categorization Engine — 3-Layer System

Layer 1 — Keyword Dictionary

```
Pre-built map of 200+ keywords → categories.
Example: 'rice', 'bread', 'milk' → Food | 'shampoo', 'soap' → Toiletry
'petrol', 'fuel', 'diesel' → Car | 'panadol', 'amoxicillin' → Health
Returns highest-priority match with confidence score 0-100.
```

Layer 2 — User History

```
If this merchant name / item name was previously categorized by this user,
reuse that mapping (stored in keyword_dictionary with is_user_defined = 1).
User-defined entries always have priority = 10 (highest).
```

Layer 3 — Merchant Context Bias

```
If merchant type is identified (pharmacy, supermarket, restaurant, gas station),
apply a category bias: pharmacy → +20 pts to Health for all items.
Final score = keyword_score + history_bonus + merchant_bias.
```


7.3 AI Integration & Fallback Strategy

On-Device (Always available)

- Google ML Kit OCR — receipt text extraction
- Statistical budget calculation — weighted moving average
- Keyword dictionary categorization — 200+ built-in keywords
- All analytics and chart generation

Optional Cloud AI (When online)

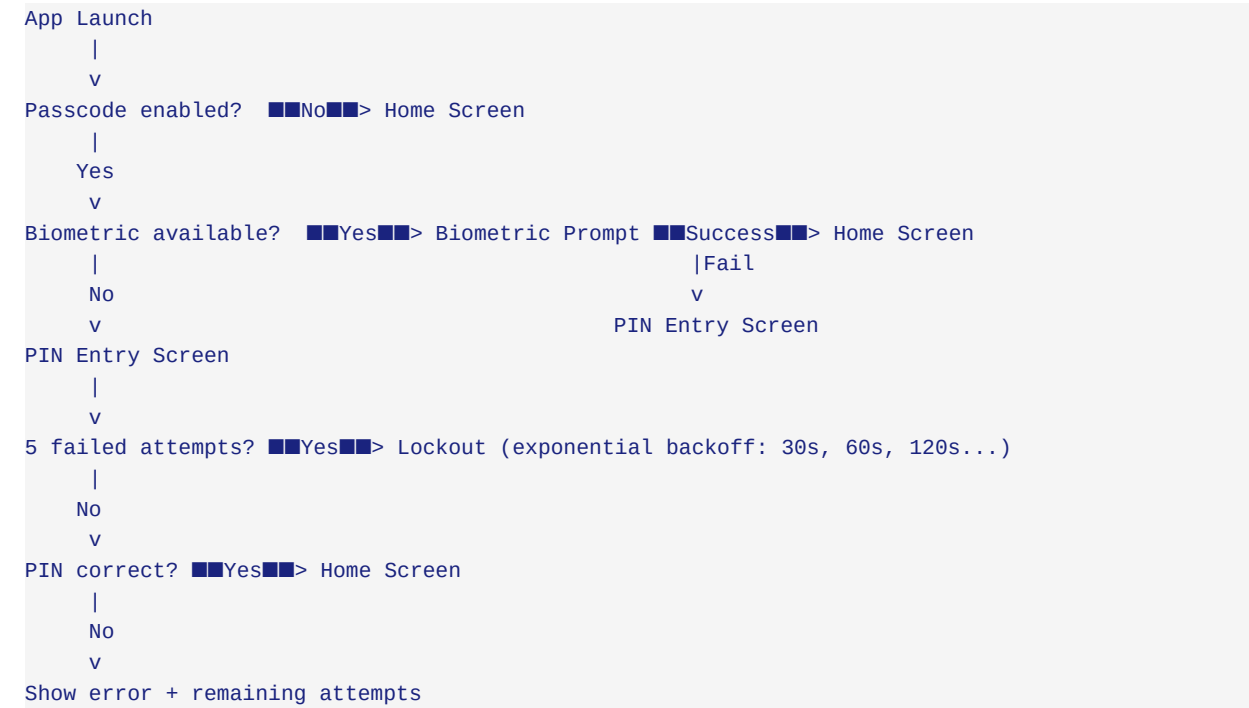
- Gemini API / OpenAI GPT — natural language plan suggestions
- Enhanced item description understanding for unusual receipts
- Budget coaching tips: 'You spent 40% on Food last month, consider...'
- Exchange rate updates from live API

8. SECURITY DESIGN

8.1 Data Encryption

Asset	Encryption Method
Database	AES-256 via SQLCipher — transparent encryption, no code changes needed
Receipt Images	Stored in app-private encrypted storage (iOS Secure Enclave / Android Keystore)
Encryption Key	Derived and stored in iOS Keychain / Android Keystore — never in plain text
Cloud Backups	Encrypted locally before upload — server never sees unencrypted data
API Keys	Stored in flutter_secure_storage — never hardcoded in source

8.2 Authentication Flow



8.3 Secure API Communication

- All outbound API calls use **HTTPS / TLS 1.3** — no HTTP allowed
- Certificate pinning for Moneyora's own backend services (if any added in future)
- API keys stored in **flutter_secure_storage** — never in source code or assets
- User financial data is **never transmitted** without explicit user action (e.g. enabling cloud sync)
- All API calls send only **statistical summaries** (category averages) — never raw transaction data
- OAuth 2.0 used for Google Drive / Dropbox authentication

9. COMPONENT INTERFACES

9.1 Repository Interface Contracts

```
// Domain Layer – Abstract Repository Interfaces (Pure Dart)

abstract class TransactionRepository {
  Future> addTransaction(Transaction t);
  Future>> getTransactions(TransactionFilter filter);
  Future> updateTransaction(Transaction t);
  Future> deleteTransaction(int id);
  Stream> watchTransactions(TransactionFilter filter);
}

abstract class MoneyPlanRepository {
  Future> generatePlan(PlanConfig config);
  Future> savePlan(MoneyPlan plan);
  Future>> getSavedPlans();
  Future> getActivePlan();
  Stream watchPlanProgress(int planId);
}

abstract class ReceiptRepository {
  Future> scanReceipt(File imageFile);
  Future>> confirmScan(ReceiptScan scan);
  Future>> getScanHistory();
}
```

9.2 Use Case Interface Contracts

```
// Base Use Case Interface
abstract class UseCase {
  Future> call(Params params);
}

// Example Use Case Implementations
class AddTransaction implements UseCase {
  final TransactionRepository repository;
  AddTransaction(this.repository);

  @override
  Future> call(TransactionParams params) async {
    // Validate business rules here
    if (params.amount <= 0) return Left(ValidationFailure('Amount must be positive'));
    return repository.addTransaction(params.toTransaction());
  }
}

class GenerateMoneyPlan implements UseCase {
  final MoneyPlanRepository repository;
  GenerateMoneyPlan(this.repository);

  @override
  Future> call(PlanConfig config) =>
    repository.generatePlan(config);
}
```

9.3 External API Interfaces

Service		Endpoint	Data Sent
Gemini	API (Optional)	POST https://generativelanguage.googleapis.com/v1/models/gemini-pro:generateContent	Budget plan enhancement suggestions — sends only category averages
Exchange Rate	API (Optional)	GET https://api.exchangerate.host/latest?base=LKR	Currency conversion rates — cached 24h, fallback to last cached value
Google Drive	API (Optional)	Google Drive REST API v3	Cloud backup upload/download — user must explicitly enable
Dropbox	API (Optional)	Dropbox API v2	Alternative cloud backup — user must explicitly enable

10. TECHNOLOGY STACK DETAILS

10.1 Core Dependencies

Package	Version	Purpose
flutter	3.x (stable)	Core framework — iOS & Android from single codebase
dart	3.0+ (null safety)	Language — strict null safety enforced
flutter_riverpod	^2.4.0	State management and dependency injection
sqlite_sqldcipher	^2.2.0	AES-256 encrypted SQLite database
path_provider	^2.1.0	App directory paths for database and files
google_mlkit_text_recognition	^0.11.0	On-device OCR for receipt scanning
fl_chart	^0.66.0	Donut charts, bar charts, line trend charts
flutter_local_notifications	^17.0.0	Budget alerts and recurring reminders
image_picker	^1.0.7	Camera and gallery image selection
local_auth	^2.1.8	Biometric authentication (Fingerprint / Face ID)
flutter_secure_storage	^9.0.0	Secure key/value storage (Keychain/Keystore)
go_router	^13.0.0	Declarative navigation and deep linking
dartz	^0.10.1	Functional Either type for error handling
http	^1.2.0	HTTP client for optional API calls
share_plus	^7.2.0	File sharing/export functionality
intl	^0.19.0	Internationalization and number/date formatting
firebase_crashlytics	^3.4.0	Production crash reporting
flutter_localizations	SDK included	Multi-language support framework

10.2 Flutter Project Setup Commands

```
# 1. Create Flutter project
flutter create --org com.moneyora --project-name moneyora moneyora

# 2. Navigate to project
cd moneyora

# 3. Add all dependencies to pubspec.yaml, then:
flutter pub get

# 4. Verify setup
flutter doctor
```

```
flutter analyze
flutter test

# 5. Run on connected device
flutter run

# 6. Build release APK (Android)
flutter build apk --release

# 7. Build release IPA (iOS)
flutter build ipa --release
```

10.3 pubspec.yaml Dependencies Section

```
dependencies:
  flutter:
    sdk: flutter
  flutter_localizations:
    sdk: flutter

  # State Management
  flutter_riverpod: ^2.4.0

  # Database
  sqflite_sqlcipher: ^2.2.0
  path_provider: ^2.1.0
  path: ^1.9.0

  # AI & ML
  google_mlkit_text_recognition: ^0.11.0
  http: ^1.2.0

  # UI & Charts
  fl_chart: ^0.66.0
  go_router: ^13.0.0

  # Device Features
  image_picker: ^1.0.7
  local_auth: ^2.1.8
  flutter_local_notifications: ^17.0.0
  flutter_secure_storage: ^9.0.0

  # Utilities
  dartz: ^0.10.1
  intl: ^0.19.0
  share_plus: ^7.2.0

  # Monitoring
  firebase_crashlytics: ^3.4.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  mockito: ^5.4.4
  build_runner: ^2.4.8
  flutter_lints: ^3.0.1
```

